Spring Security

Complete Learning Notes — Part 2

1. What is the http Variable in SecurityConfig

HttpSecurity is a class provided by Spring Security. When you write your SecurityFilterChain method, Spring Security automatically passes you an object of this class. That object is what you use to configure all the security rules.

Think of it like this. Spring Security hands you a settings panel and says here, use this to tell me how you want security to work. That settings panel is the HttpSecurity object. You call methods on it like csrf disable or sessionManagement or authorizeHttpRequests and each one is basically you clicking a setting on that panel.

The variable is named http just by convention. You could name it anything. But developers name it http because it represents the configuration of how your application handles incoming HTTP requests.

> *http is not something mysterious. It is just an object Spring Security gives you that represents all the configurable security behavior of your application. You use it to tell Spring Security exactly how you want things to work instead of using its defaults.*

2. Why Securing Anything Was Needed at All

When the internet started, websites were simple. Just pages of text and images. Anyone could read them. There was nothing personal, nothing sensitive, no accounts, no data. Security was not a concern because there was nothing to protect.

Then websites started becoming applications. Banks came online. Emails came online. Shopping came online. Suddenly your personal data, your money, your private messages were sitting on a server somewhere. And anyone who could send an HTTP request to that server could potentially access it.

So the need arose to answer one basic question before processing any request. Who are you and should I trust you. That is the entire reason security exists.

3. How Authentication Evolved — The Full Story

Stage 1 — Basic Auth (The Beginning)

The very first and simplest solution was to just attach your username and password to every request. Server checks them every time. If correct, process the request. If wrong, reject it. This is httpBasic.

But it had serious problems. Your username and password travel with every single request every single time. If anyone intercepts even one request they have your credentials forever. There is no expiry. There is no way to say this access is only valid for one hour. You either have

the password or you do not. Changing access means changing the password. It is like giving someone the key to your house and the only way to stop them is to change the lock.

So this worked for very simple internal systems but was clearly not good enough for real user facing applications.

Stage 2 — Sessions and Cookies

As websites grew more complex, developers thought of a better approach. Instead of sending credentials every single time, what if the user proves themselves once and the server remembers them for a while. This is how sessions were born.

User sends username and password once. Server verifies them. Server creates a session, which is basically a record stored in the server's memory that says user John is authenticated and his session ID is abc123. Server sends that session ID to the browser as a cookie. Browser stores it automatically. Every request after that just sends the cookie. Server looks up the session ID, finds John in its records, and knows who he is without asking for credentials again.

This was a huge improvement. Credentials only travel once. After that just a session ID. Much safer. This became the standard for years. Every website in the 2000s and early 2010s worked exactly like this.

But this also had problems that became more visible as applications grew. The server has to store every session in memory or database. If you have a million users logged in at the same time, the server is storing a million sessions. If the server crashes, all sessions are lost and everyone gets logged out. If you want to scale your application across multiple servers, you have a problem because server A has the session but the user's request might go to server B which has no idea who they are. Also CSRF attacks became possible because of this cookie based approach.

Stage 3 — Tokens and JWT

As applications grew even more complex, especially with mobile apps and multiple frontends, developers needed something better. Something where the server does not need to remember anything. Something that works the same whether the user is on a browser, a mobile app, or another service calling your API.

The idea was simple. Instead of the server remembering who you are, what if we give you a small document that proves who you are, and you carry that document yourself and show it whenever needed. This is JWT — JSON Web Token.

User sends username and password once to a login endpoint. Server verifies them. Instead of creating a session, the server creates a small encoded string that contains your user ID, your role, and an expiry time, and signs it with a secret key. This signed string is the JWT token. Server sends it back to you and forgets about you completely.

You store the token yourself in your React app's memory. Every request you manually attach it in the Authorization header. Server receives the request, reads the token, verifies the signature to make sure nobody tampered with it, reads your user ID and role from inside the token, and processes the request. No session lookup. No database call to check who you are. Everything the server needs is inside the token itself.

This solved all the problems of sessions. Server stores nothing. Scales perfectly across multiple servers because every server can verify the token independently. No CSRF risk because tokens are in headers not cookies and browsers never automatically send headers. Works identically for browser apps, mobile apps, and service to service communication.

4. Why Old Things Are Not Needed Anymore

formLogin is not needed because there is no session to create after login. The whole point of formLogin was to establish a session. Without sessions it has no purpose.

Sessions are not needed because the token itself carries everything the server needs. The server does not need to remember you.

CSRF protection is not needed because you are not using cookies for authentication. The entire CSRF problem existed because of automatic cookie sending. Without cookies in the authentication flow, CSRF attacks cannot work.

httpBasic is not needed in production because sending raw credentials on every request is less secure than sending a token that expires. But it stays useful for quick testing and learning.

5. The Authorization Header — What It Is and How It Evolved

In the earliest days with httpBasic, credentials traveled in the Authorization header like this.

```
Authorization: Basic dXNlcjpwYXNzd29yZA==
```

That encoded string is just your username and password combined and encoded. Server decodes it and checks.

With JWT the same header is used but now carries a token instead of credentials.

```
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
```

The word Bearer just means I am the bearer of this token, here it is. Server reads it, verifies it, and knows who you are.

So the Authorization header is not a new concept. It was always there from the httpBasic days. JWT just uses the same header in a much better way. Instead of raw credentials it carries a secure signed token.

6. formLogin vs httpBasic — The Difference

Your understanding is exactly right. Credentials used in a form in the browser is formLogin and same credentials entered in Postman in the header is httpBasic.

formLogin

This is specifically designed for the traditional web app world. It gives you a login page rendered by the server. You fill the form, submit it, the server creates a session, the browser stores a cookie, and from that point the session cookie is what proves who you are on every request. The end result of formLogin is always a session.

httpBasic

This does not care about sessions at all. It does not try to create one. It does not need one. Every single request just carries the credentials directly in the header and the server checks them right there on that request and either allows or rejects it. No session involved at any point. This is perfectly compatible with stateless.

Why mentor removed formLogin but kept httpBasic

Your mentor already set SessionCreationPolicy to STATELESS which means he told Spring Security to never create sessions under any circumstance. So even if someone successfully logs in through the form, no session gets created after that. The user would be stuck. Logged in but with nothing to prove it on the next request. formLogin becomes completely pointless because the one thing it was supposed to create, which is a session, is now disabled. It is like a door that opens into a room that does not exist.

httpBasic on the other hand works perfectly without sessions. So he kept httpBasic purely as a temporary way to test APIs through Postman while JWT is not set up yet. Once JWT arrives in Section 22, httpBasic also goes away and the Authorization header will carry a token instead of raw credentials.

7. How the Server Checks Credentials

In the current learning project

Right now your mentor is using Spring Security's default in-memory user. When the application starts, Spring Security automatically creates one user in memory with username user and a randomly generated password that prints in the console. When you send a request with httpBasic, Spring Security takes the username and password from the header, compares them against this in-memory user, and if they match it allows the request. This is only for learning purposes. The user exists only in memory and disappears the moment the server restarts.

In a real application

In a real application credentials are stored in a database. You have a users table with columns like ID, username, and password. The password is never stored as plain text. It is stored in an encrypted form using something called BCrypt. When a request comes in with credentials, Spring Security takes the username, looks it up in the database, finds the stored encrypted password, and then uses BCrypt to check if the password you sent matches the encrypted one stored. If yes it allows the request. If no it rejects it.

How Spring Security knows where to look

Spring Security does not automatically know about your database. You have to tell it by creating something called a UserDetailsService. This is a class where you tell Spring Security how to find a user. You write the logic to look up the user from your database by username and return their details. Spring Security then uses those details to verify the credentials. Your mentor covers this in lecture 307.

Complete flow in a real application

Someone sends a request with credentials. Spring Security intercepts it. It takes the username and calls your UserDetailsService to fetch that user from the database. It gets back the user object with the encrypted password. It uses BCrypt to compare the incoming password with the stored encrypted one. If they match the request is authenticated and processed. If not it returns 401 unauthorized.

8. SecurityConfig — Three Ways to Write the Same Thing

All three styles do exactly the same thing. They configure Spring Security to disable CSRF, require authentication on every request, enable httpBasic, and make sessions stateless. The only difference is how they are written.

Style 1 — Imperative (Without Lambda)

In this style for every single configuration you create a full separate Customizer object, write an @Override method inside it, put the logic there, and pass that object to http. This is the old verbose way of writing configuration in Java before lambdas existed.

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {

    Customizer<CsrfConfigurer<HttpSecurity>> custCsrf = new
    Customizer<CsrfConfigurer<HttpSecurity>>() {
        @Override
        public void customize(CsrfConfigurer<HttpSecurity> configurer) {
            configurer.disable();
        }
    };
    http.csrf(custCsrf);

    Customizer<AuthorizeHttpRequestsConfigurer<HttpSecurity>
    .AuthorizationManagerRequestMatcherRegistry> custHttp = new
    Customizer<AuthorizeHttpRequestsConfigurer<HttpSecurity>
    .AuthorizationManagerRequestMatcherRegistry>() {
        @Override
        public void customize(AuthorizeHttpRequestsConfigurer<HttpSecurity>
        .AuthorizationManagerRequestMatcherRegistry registry) {
            registry.anyRequest().authenticated();
        }
    };
    http.authorizeHttpRequests(custHttp);

    return http.build();
}
```

Just to disable CSRF and secure all endpoints he had to write all of this. Every configuration needs its own object, its own override, its own method. Very long and repetitive for something so simple.

Style 2 — Lambda (http on every line)

In this style the arrow function replaces that entire object creation and override pattern. Instead of creating a separate object you write the logic inline right where it is needed using the arrow. Java introduced lambdas specifically to replace this verbose pattern and make code shorter and cleaner.

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {

    http.csrf(customizer -> customizer.disable());
    http.authorizeHttpRequests(request ->
request.anyRequest().authenticated());
    http.httpBasic(Customizer.withDefaults());
```

```
    http.sessionManagement(session -> session
        .sessionCreationPolicy(SessionCreationPolicy.STATELESS));

    return http.build();
}
```

Here customizer -> customizer.disable() is doing the exact same thing as that entire custCsrf object with override and method. Take the customizer object Spring gives you and call disable on it right here. One line instead of ten. Same with request -> request.anyRequest().authenticated(). Take the request object and apply the rule right here inline. One line instead of ten.

http is still written on every line separately and every line ends with a semicolon because each is still a separate statement.

Style 3 — Lambda with Builder Pattern (Final Code)

Builder pattern means you configure one object step by step by chaining methods one after another. Each method does its job and returns the same http object back so you can keep calling the next method on it using a dot. That is why http is written only once at the top and removed from every other line. That is why semicolons are removed from every line except the last one. Because this is no longer multiple separate statements. It is one single continuous chain configuring one object.

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {

    http.csrf(customizer -> customizer.disable())
        .authorizeHttpRequests(request ->
request.anyRequest().authenticated())
        .httpBasic(Customizer.withDefaults())
        .sessionManagement(session -> session
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS));

    return http.build();
}
```

This is the final and most modern way. This is what you will see in every real Spring Boot project at a job.

Side by Side — Just for CSRF

Imperative style to disable CSRF.

```
Customizer<CsrfConfigurer<HttpSecurity>> custCsrf = new
Customizer<CsrfConfigurer<HttpSecurity>>() {
    @Override
    public void customize(CsrfConfigurer<HttpSecurity> configurer) {
        configurer.disable();
    }
};
http.csrf(custCsrf);
```

Lambda style to disable CSRF.

```
http.csrf(customizer -> customizer.disable())
```

> *Identical result. Completely different amount of writing.*

Why Mentor Showed All Three

Because in older Spring Boot projects and older tutorials online you will see the imperative style. If your mentor only showed you lambda style and you opened an old codebase at a job and saw all those Customizer objects and overrides you would be completely confused. Now you know all three styles do the same thing and you can read any of them comfortably wherever you see them.

> ***Next up — Working with Multiple Users, UserDetailsService, BCrypt, and Database Integration.***